myPortfolio — Global Stock Portfolio & Historical Wealth Tracker (`v2.4.0` • Build `20261018_2345`)

myPortfolio is a client-side global stock portfolio and historical wealth tracker supporting Hong Kong (`HKEX .HK`), United States (`NYSE / NASDAQ`), and London (`LSE .L`) equities. It features a compartmentalized JavaScript architecture (`version.js`, `logger.js`, `utils.js`, `api.js`, `portfolio.js`, `charts.js`, `app.js`) that is also pre-bundled inside `index.html` for zero-build static deployment.

Release `v2.4.0` (Build `20261018_2345`) — Highlights

1. **Removed All Hardcoded/Fake `hardcoded market snapshot data` (`api.js`, `portfolio.js`, `index.html`, `charts.js`):**
   - Completely removed `hardcoded market snapshot data` from `api.js`, `portfolio.js`, and `index.html`.
   - Added `isLegacySyntheticHistory()` in `portfolio.js` to automatically purge any legacy synthetic price histories cached in browser `localStorage` or old `.xlsx` backups.
   - Updated `computeMonthlyPortfolioSeries()`, `renderMonthlyHistoryChart()`, and `renderInlineStockChart()` in `charts.js` so charts remain strictly empty whenever no real historical price series is available.
2. **Startup Loading of `./historical_prices.json` (`portfolio.js`):**
   - `initAppState()` automatically calls `loadHistoricalPricesFromJson()` on startup to fetch `./historical_prices.json` (`cache: 'no-store'`), ingesting real monthly OHLCV + adjusted close series, quotes, and fundamentals into application state and re-rendering all charts.
3. **Sector-Grouped Holdings with Collapsible Headers, ETFs at Top, and Alphabetical Sorting (`app.js`, `portfolio.js`, `styles.css`):**
   - `renderHoldingsTable()` in `app.js` groups active holdings by sector with interactive **collapsible sector header rows** showing position counts, portfolio weight %, and sector subtotals (Total Cost, Market Value, Unrealized P&L, Yield on Cost) plus one-click **Expand All / Collapse All** controls.
   - **ETFs (`Index ETF`)** are always pinned at the top of the Holdings table, followed by equity sectors sorted **alphabetically (`A → Z`)**, with tickers inside each sector sorted **alphabetically (`A → Z`)**.
4. **Automated Yahoo Finance v8 Monthly Pipeline (`historical_prices.json`, `update_historical_prices.py`, `.github/workflows/update-historical-prices.yml`):**

- `update_historical_prices.py` fetches 10-year monthly adjusted & unadjusted closing prices, highs, lows, volumes, live quotes, and trailing 12M dividend yields from Yahoo Finance's v8 Chart API (`query1` / `query2.finance.yahoo.com`) and writes `./historical_prices.json`.
- `.github/workflows/update-historical-prices.yml` runs `update_historical_prices.py` daily via GitHub Actions (`cron: '30 22 * * *'` and manual `workflow_dispatch`) and commits updated `historical_prices.json` data back to the repository.

Compartmentalized Project Structure

File	Version	Build Stamp	Role & Responsibility
`index.html`	`v2.4.0`	`20261018_2345`	Self-contained HTML5 application containing layout, Chart.js canvases, modals, and all 7 embedded JS modules
`api.js`	`v2.4.0`	`20261018_2345`	Live Global `MarketAPI` client (Yahoo v8 proxy race, Tencent JSONP, keyed providers; zero fake fallback data)
`portfolio.js`	`v2.4.0`	`20261018_2345`	Startup `./historical_prices.json` loader, legacy synthetic cache scrubber, WACB/P&L/DRIP engine & SheetJS Excel I/O
`app.js`	`v2.4.0`	`20261018_2345`	Collapsible sector-grouped Holdings table (ETFs at top, A–Z sector &

File	Version	Build Stamp	Role & Responsibility
			ticker sorting), modals & event listeners
`charts.js`	`v2.4.0`	`20261018_2345`	Chart.js controllers for Portfolio Balance & Return + inline stock charts (strictly empty when no real data is loaded)
`logger.js`	`v2.4.0`	`20261018_2345`	Real-time runtime log & error interceptor, console hook, and 8-point Page & DOM Health Check suite
`utils.js`	`v2.4.0`	`20261018_2345`	Financial precision arithmetic (`Fin`) and automatic client `DeviceDetector`
`version.js`	`v2.4.0`	`20261018_2345`	Centralized version manifest, module registry, and release changelog
`styles.css`	`v2.4.0`	`20261018_2345`	Responsive layout rules, collapsible sector header styles, mobile 7-tab bottom-nav safe areas & sticky table columns
`historical_prices.json`	`v2.4.0`	`20261018_2345`	External JSON dataset loaded by `portfolio.js` on startup and populated by

File	Version	Build Stamp	Role & Responsibility
			`update_historical_prices.py`
`update_historical_prices.py`	`v2.4.0`	`20261018_2345`	Standalone Python 3 Yahoo Finance v8 monthly historical price fetcher script
`.github/workflows/update-historical-prices.yml`	`v2.4.0`	`20261018_2345`	Daily GitHub Action workflow that runs `update_historical_prices.py` and commits `historical_prices.json`
`myPortfolio_Yahoo_Restore_20261018.xlsx`	`v2.4.0`	`20261018_2345`	6-sheet Excel backup workbook with 22 HKEX holdings & 78 share lots (synthetic `Historical_Prices` rows removed)

HTTP 429 (`Too Many Requests`) Protection on GitHub Actions & Local Runs

Yahoo Finance's edge (`ATS`) fingerprints standard Python `urllib` / `requests` TLS handshakes (`JA3`/`JA4`) from datacenter IPs (such as GitHub Actions `ubuntu-latest` runners) and blocks rapid or un-cookie'd connections with `HTTP Error 429: Too Many Requests`.

`update_historical_prices.py` and `.github/workflows/update-historical-prices.yml` prevent this using a 3-tier strategy:

1. **Browser TLS Impersonation (`curl_cffi`) + Yahoo Crumb/Cookie Handshake**: Installed automatically in the GitHub Action workflow (`pip install --upgrade curl_cffi yfinance openpyxl`). Uses `curl_cffi` to impersonate Chrome/Safari/Edge TLS handshakes, warms up Yahoo session cookies (`fc.yahoo.com`), and attaches a valid `/v1/test/getcrumb` token to `/v8/finance/chart` requests.

2. **Automatic Session Rotation, `Retry-After` Backoff & Jittered Pacing**: Spaces requests by `1.25s + 0.0–0.6s` random jitter (`--delay 1.25 --jitter 0.6`), rotates TLS profiles on `HTTP 429`, honors `Retry-After` headers, and retries up to 5 times with exponential backoff.
3. **`yfinance` Library Fallback & Cache Preservation**: Falls back to `yfinance.Ticker(symbol).history()` if direct `v8/finance/chart` requests still fail for an individual symbol, while preserving any previously cached series in `historical_prices.json`.